

SUMMER INTERNSHIP REPORT

ON

Product Development

Submitted by

Mettu Mani Chandhan Sai

UNDER THE SUPERVISION

OF

Arun kumar R



**In the Department of
Advance Engineering**

Organization

Ashok Leyland Technical Centre

Vellivoyal Chavadi

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Ashok Leyland** Limited for providing me with the opportunity to undertake my internship at the Technical Centre. This internship has been an invaluable learning experience, allowing me to gain practical exposure to the automotive industry and understand the product development process in a professional engineering environment.

I extend my heartfelt thanks to the **Advanced Engineering Department**, particularly the Software Development team **Manager Murali Manohar**, for their continuous guidance, support, and encouragement throughout my internship. Working with the team enabled me to enhance my technical knowledge, understand industry practices, and develop a better appreciation of automotive software development and its role in modern vehicle systems.

I would like to express my sincere appreciation to my mentors **Mr. Sundhareshan**, **Mr. Dheenadayalan**, **Mr. Ajay** and **Mr. Arun Kumar**, for their valuable guidance, constant support, and willingness to share their knowledge and experience. Their mentorship helped me understand technical concepts, complete my assigned tasks effectively, and gain confidence in applying engineering principles in a professional work environment. Their encouragement and constructive feedback were instrumental in making my internship a rewarding learning experience.

I am also grateful to all the engineers and staff members of the **Software Development** team for their cooperation, assistance, and friendly work environment. Their support made it possible for me to learn about various engineering processes and gain practical insights into the development of automotive software.

Finally, I would like to thank my faculty members and my institution for giving me the opportunity to undertake this internship as a part of my academic

curriculum. The knowledge and experience gained during this training will greatly contribute to my professional growth and future career in engineering.

ABSTRACT

This report presents the work carried out during an industrial internship at **Ashok Leyland** Technical Centre, Vellivoyal Chavadi, within the Advanced Engineering Department's Software Development team, focusing on Light Commercial Vehicles (LCVs). The internship began with an orientation to Ashok Leyland's broader product development ecosystem, including exposure to Simulation Analytics, Cyber Security, Functional Safety, Embedded Software, and Software Development teams.

Within the Software Development team, the core assignment involved observing and manually testing **ASCEND**, an internal project-tracking platform, to build familiarity with its dashboard, sprint tracking, and merge request workflows. Structured manual testing was carried out against a QA checklist, surfacing UI, fallback-state, loading-state, and validation defects that were logged, fixed, and re-verified — several of which were folded back into the checklist as permanent test cases. Alongside testing, the project's frontend codebase was studied to understand real-world usage of React with TypeScript, Vite, SCSS, and MUI, culminating in the practical implementation of four original entrance-animation effects for the ASCEND dashboard.

Building on this foundation, an independent project titled Vordenk was conceptualized and developed to address a diagnostic gap in commercial vehicle fault analysis: the lack of pre-fault behavioural context behind Diagnostic Trouble Codes (DTCs). Vordenk introduces the concept of a " **fault fingerprint** " — a fleet-wide, pre-fault trend record for RPM, temperature, load, and speed — built using

a three-role (Tester–Analyzer–Integrator) workflow, a React/Vite/SCSS frontend, a Supabase backend, and the Lovable AI gateway for AI-assisted root-cause suggestions. The report documents Vordenk's system design, technology stack, implementation details, challenges encountered, and scope for future enhancement.

Overall, the internship provided hands-on grounding in both quality assurance practice and frontend engineering, and demonstrated the ability to translate that learning into an original, independently architected solution to a real industry problem.

Table of Contents

Acknowledgement

Abstract

1. Introduction to Ashok Leyland

1.1 Advanced Engineering Department

1.1.1 Simulation Analytics

1.1.2 Cyber Security

1.1.3 Functional Safety

1.1.4 Embedded Software

1.1.5 Software Development

2. Project Observation — ASCEND

2.1 About Ascend

2.2 Motive Behind Building ASCEND

2.3 How ASCEND Is Used

2.4 Introduction to Manual Testing

2.5 Manual Testing Experience on the Ascend Project

2.6 Key Learnings from Frontend Observation

2.6.1 React, TSX, and Vite.js

2.6.2 SCSS

2.6.3 MUI (Material UI) Components

2.7 Practical Implementation: Intro Animation for the Ascend Project

3. Vordenk — Digital Fleet Fingerprint

3.1 Introduction to Vordenk

3.2 Fundamentals of Diagnostic Trouble Codes (DTCs)

3.2.1 How Is a DTC Code Structured?

3.2.2 How to Read Diagnostic Trouble Codes?

3.2.3 What to Look for in a Vehicle Diagnostic Tool

3.2.4 Why Fault Code Analysis Matters for Fleet Management

3.2.5 Common Examples of Diagnostic Trouble Codes

3.3 Problem Statement

3.4 Proposed Solution

3.4.1 Core Idea

3.4.2 How Vordenk Solves the Problem

3.5 Central Work Idea

3.5.1 System Workflow

3.5.2 Role-Based Workflow (Tester, Analyzer, Integrator)

3.5.3 Feature Mapping per Role

3.6 Technology Stack

3.6.1 Frontend — React, Vite, SCSS

3.6.2 Backend — Supabase

3.6.3 AI Layer — Lovable AI Gateway

3.6.4 Why This Stack Was Chosen

3.7 Implementation

3.7.1 React Implementation — Component Structure & Page Control

3.7.2 SCSS Implementation

3.7.3 Supabase Implementation

3.8 Workflow Diagrams

4. Challenges Faced

5. Future Scope

6. Feedback

7. References

1.INTRODUCTION TO ASHOK LEYLAND

Ashok Leyland Limited is one of India's largest and most respected manufacturers of commercial vehicles. Established in 1948 and headquartered in Chennai, Tamil Nadu, the company is the flagship automotive business of the Hinduja Group. Over the years, Ashok Leyland has built a strong reputation for designing and manufacturing trucks, buses, light commercial vehicles, defence vehicles, engines, and power solutions. The company serves both domestic and international markets and has a significant presence across Asia, Africa, the Middle East, and several other regions. Its products are widely recognized for their durability, reliability, fuel efficiency, and technological advancements.

Research and development have always been one of the major strengths of Ashok Leyland. The company's Technical Centre houses several specialized departments that work together to design, develop, validate, and improve commercial vehicles before they enter mass production. Engineers from different disciplines collaborate to ensure that every vehicle meets customer expectations, safety requirements, environmental regulations, and performance

standards. Modern technologies such as computer-aided engineering, embedded electronics, software development, cybersecurity, simulation, and advanced testing methods are integrated into the vehicle development process to enhance product quality and innovation.

Ashok Leyland continuously invests in advanced manufacturing technologies and digital engineering practices to remain competitive in the global automotive industry. The organization places strong emphasis on sustainable mobility by developing fuel-efficient engines, alternative fuel technologies, electric vehicles, and intelligent transportation solutions. Continuous improvement, innovation, quality, and customer satisfaction form the foundation of the company's engineering philosophy.

The company follows a structured product development process in which every new vehicle undergoes extensive design verification, laboratory testing, prototype validation, and real-world performance evaluation before commercial production begins. Specialized departments such as the Engine Development Centre, Component Testing Laboratory, Proto Shop, Test Track, and Advanced Engineering Department play essential roles in this development cycle. Each department contributes its technical expertise to ensure that the final product meets the highest standards of safety, reliability, durability, performance, and regulatory compliance.

During my internship at Ashok Leyland, I was assigned to the Advanced Engineering Department under the Product Development division, specifically in the Software Development team. This internship provided valuable exposure to modern automotive engineering practices and enabled me to understand how software technologies are integrated with vehicle systems to support the development of next-generation commercial vehicles.



1.1 Advanced Engineering Department

The Advanced Engineering Department focuses on developing future vehicle technologies and innovative engineering solutions that enhance vehicle performance, safety, intelligence, and connectivity. This department supports next-generation product development by integrating modern automotive technologies with advanced software and electronic systems.

The department works on various aspects of vehicle innovation, including digital engineering, virtual validation, intelligent control systems, software architecture, cybersecurity, and functional safety. It collaborates closely with mechanical, electrical, and manufacturing teams to develop technologically advanced

commercial vehicles capable of meeting future market demands and regulatory requirements.

Within the Advanced Engineering Department, Product Development comprises several specialized teams that collectively contribute to modern vehicle development.

1.1. Simulation & Analytics

The Simulation & Analytics team uses advanced computer-aided engineering (CAE) tools to perform virtual simulations during the product development process. Instead of relying solely on physical prototypes, engineers analyze structural strength, thermal behavior, crash performance, fluid flow, vibration characteristics, and durability through computational models. Virtual simulation significantly reduces development time and cost while enabling engineers to optimize designs before physical testing begins. The results obtained from simulations assist in making informed engineering decisions and improving overall product performance.

1.1.2 Cyber Security

The Cyber Security team focuses on protecting modern vehicles from digital threats and unauthorized access. As commercial vehicles increasingly incorporate connected technologies, telematics, electronic control units, wireless communication, and software-based functionalities, ensuring cybersecurity has become an essential aspect of vehicle development.

The department implements secure software architectures, encryption techniques, authentication mechanisms, secure communication protocols, and vulnerability assessment procedures to safeguard vehicle systems. Engineers perform security testing and risk analysis to identify potential cyber threats and ensure compliance with international automotive cybersecurity standards,

thereby enhancing vehicle safety and data protection.

1.1.3 Functional Safety

The Functional Safety department ensures that electrical and electronic systems operate safely even in the presence of faults or failures. Engineers identify potential hazards, evaluate associated risks, and design safety mechanisms that minimize the possibility of accidents resulting from system malfunctions.

Following internationally recognized automotive functional safety standards, the department performs hazard analysis, risk assessment, safety requirement development, verification, validation, and safety testing throughout the product development lifecycle. Functional Safety plays a vital role in developing reliable electronic control systems that maintain safe vehicle operation under both normal and fault conditions.

1.1.4 Embedded Software

The Embedded Software department develops software that directly controls electronic hardware within the vehicle. Modern commercial vehicles contain numerous Electronic Control Units (ECUs) responsible for managing engine control, braking systems, transmission, body electronics, diagnostics, instrument clusters, and various intelligent vehicle functions.

Engineers design, implement, integrate, and validate embedded software using industry-standard programming languages, model-based development techniques, and automotive communication protocols. The department ensures that embedded software operates efficiently, reliably, and in compliance with automotive quality and safety standards.

1.1.5 Software Development

The Software Development department is responsible for designing, developing,

testing, and maintaining software solutions that support vehicle functionalities, engineering processes, diagnostics, and data management systems. The team develops software applications that improve vehicle performance, facilitate communication between electronic control units, automate engineering workflows, and enhance overall product development efficiency.

Software engineers employ modern programming languages, software development frameworks, version control systems, debugging tools, and testing methodologies to build robust and reliable software solutions. The department follows structured software development life cycle practices, including requirement analysis, design, implementation, testing, validation, deployment, and maintenance. Continuous collaboration with hardware engineers, validation teams, cybersecurity specialists, and functional safety engineers ensures that the developed software meets performance, quality, reliability, and regulatory requirements.

During my internship, I was assigned to the Software Development team within the Product Development division of the Advanced Engineering Department. This provided me with the opportunity to gain practical exposure to software engineering practices in the automotive industry, understand the software development lifecycle used for vehicle applications, and observe how software integrates with embedded systems and electronic control units to support modern commercial vehicle technologies.

2. PROJECT OBSERVATION

2.1 About Ascend

ASCEND is an internal software platform used at Ashok Leyland to help teams track, manage, and visualize ongoing project and development activity in a single, centralized dashboard. Rather than relying on scattered spreadsheets, disconnected tools, or manual status updates passed between teams, ASCEND consolidates key project information — sprint progress, ticket status, merge requests, notifications, and team activity — into one interface that gives every stakeholder a real-time view of where a project stands.

At its core, ASCEND is built around a dashboard that surfaces the health of a project at a glance. Key performance indicators such as the current software version, the upcoming version in development, the number of open issues, and overall sprint progress are displayed prominently, giving managers and developers an immediate sense of project status without needing to dig through individual tickets or ask around for updates. Beyond the summary view, ASCEND includes a calendar view for tracking sprint timelines and milestones, a merge request tracker for monitoring code changes moving through review, and a notifications system that keeps team members informed of relevant updates as they happen.

2.2 Motive Behind Building ASCEND

The motivation behind ASCEND stems from a common challenge in growing engineering teams: as the number of active projects, sprints, and contributors increases, keeping everyone aligned on progress becomes increasingly difficult

without a unified system. Before a tool like ASCEND, teams often had to rely on manual check-ins, isolated tracking sheets, or fragmented communication across emails and chat threads to understand where a project stood. This made it harder to catch delays early, track accountability for open issues, or give new team members a quick, accurate picture of ongoing work.

ASCEND was conceived to solve this by acting as a single source of truth — a live, visual representation of project health that anyone on the team could open and immediately understand. By combining sprint tracking, issue management, merge request visibility, and team activity into one dashboard, ASCEND reduces the overhead of manually compiling status updates and instead lets that information flow naturally from the work itself. The long-term goal is to make project visibility effortless, so that identifying bottlenecks, tracking release readiness, and coordinating across teams becomes a byproduct of using the tool day-to-day, rather than a separate administrative task.

2.3 How ASCEND Is Used

In practice, different roles interact with ASCEND in different ways:

Developers use ASCEND to track their assigned tickets, monitor merge request status as their code moves through review, and stay updated on sprint deadlines through the calendar view.

Project managers and leads rely on the dashboard's KPI summary — current version, next version, open issues, and sprint progress — to assess project health quickly and identify where attention is needed without requesting manual status reports.

QA and testing teams use ASCEND to cross-reference reported defects against ongoing sprints, ensuring that issues found during testing are properly logged, tracked, and resolved before a release.

New team members benefit from ASCEND's consolidated view, as it gives

them a fast, visual way to understand ongoing work and team activity without needing to be walked through separate tools or spreadsheets.

Because ASCEND was still in an active development stage during the course of this internship, the platform was not yet a finished, polished product — it was a working system that was actively being built out, tested, and refined by the internal team, with new features and UI improvements being added incrementally.

2.4 Introduction to Manual Testing

Manual testing is the process of verifying a software application's behaviour by executing test cases by hand, without the aid of automation scripts or tools. A tester manually walks through the application in the same way an end user would — clicking buttons, filling forms, navigating between screens, and checking outputs against expected results. Because it relies entirely on human observation, manual testing is particularly effective at catching problems that are hard to script for, such as visual inconsistencies, layout breakage, unclear error messages, and awkward user flows.

2.5 Manual Testing Experience on the Ascend Project

During my internship, I was assigned to manually test the Ascend project against a structured test checklist maintained by the QA team. Each checklist item mapped to a specific module or user flow (dashboard widgets, calendar view, merge request tracking, notifications, and so on), and my role was to execute every item, record the outcome, and log any deviation from the expected behaviour as a defect.

- **Checklist gaps:** Several scenarios were not covered by the existing checklist at all — for instance, edge cases like an empty dashboard state (no sprints assigned yet) or a user with no team avatars had no corresponding

test case, so these paths were going untested release after release.

- **Visible UI errors:** I found misaligned buttons and icons on smaller viewport widths, inconsistent spacing between cards on the dashboard, text overflowing outside its container on long project names, and colour-contrast issues on certain status badges that made them hard to read.
- **Fallback / error-state errors:** When an API call failed or returned no data, several components simply rendered blank instead of showing a proper fallback UI — for example, the activity feed showed an empty white panel instead of a "No recent updates" message, and a failed chart request left a broken layout rather than an error or retry state.
- **Loading-state gaps:** A few components jumped straight from nothing to fully-loaded content with no skeleton or spinner in between, which made the interface feel like it had frozen for a moment.
- **Navigation and validation errors:** A couple of form fields accepted invalid input without showing a validation message, and one navigation link led to a stale route left over from an earlier version of the page.

Each of these was logged as a bug with steps to reproduce, screenshots, and severity, then re-verified once a fix was pushed. This process gave me a much sharper eye for the small details that separate a functional interface from a polished, production ready one

2.6 Key Learnings from Frontend Observation

Alongside manual testing, I spent time studying the Ascend project's frontend codebase on GitLab to understand how a real, production-grade React application is structured. This section covers the core technologies I learned — React with TypeScript (TSX) on Vite, SCSS for styling, and MUI for the component library — and how observing the existing project taught me each of them.

2.6.1 React, TSX, and Vite.js

React is a JavaScript library for building user interfaces, originally developed by Meta, built around the idea of breaking an interface down into small, reusable components. Instead of manipulating the browser's DOM directly, React uses a declarative model: a developer describes what the UI should look like for a given state, and React efficiently figures out how to update the actual DOM through its virtual DOM, re-rendering only the parts that changed. This makes UIs faster to build, easier to reason about, and much easier to maintain as an application grows.

TSX is simply JSX — React's syntax that lets HTML-like markup live inside JavaScript — written with TypeScript instead of plain JavaScript. Using .tsx files means every component, prop, and piece of state is statically typed, so mismatched data types, missing props, or typos are caught by the compiler and the editor before the code ever runs, rather than surfacing as a runtime bug later.

Vite.js is the build tool and development server that the Ascend project uses to run and bundle this React + TypeScript code. Compared to older tools like Create React App or plain Webpack, Vite starts the dev server almost instantly and uses native ES modules with Hot Module Replacement, so changes made to a component appear in the browser in a fraction of a second, which made the day-to-day feedback loop of exploring and testing the codebase very quick.

Types used in the project: Studying Ascend's code showed me how TypeScript types are actually used in practice, rather than just in theory:

- **Interfaces / type aliases** to describe the shape of data, such as a Sprint, Ticket, or User object returned from the API.
- **Prop types** for every component, defining exactly what data a component expects and whether each field is required or optional.
- **State types** , used with hooks like `useState<Ticket[]>()` so that the state variable is guaranteed to always hold the correct shape of data.
- **Generics and utility types** (e.g. `Partial<>`, `Record<>`) used in shared hooks

and API-response wrappers to keep code reusable without losing type safety.

Components: The project follows a component-based structure, and observing it taught me:

- How the codebase separates **functional components** (built with hooks) into folders such as components/, pages/, hooks/, and types/, keeping presentation, logic, and data shapes cleanly separated.
- How **hooks** like useState, useEffect, and useContext are used to manage local state, handle side effects such as data fetching, and share data across components without prop-drilling.
- How small, reusable components (cards, tables, modals, buttons) are composed together to build entire pages like the dashboard and calendar view.

I learned all of this largely by investigation — cloning the Ascend repository from GitLab, reading through existing components file by file, tracing how data flowed from an API call into a hook and finally into the JSX returned by a component, and cross-referencing merge requests to see how new features had been added on top of this structure. This hands-on exploration taught me far more about real-world React patterns than reading documentation alone would have.

2.6.2 SCSS

SCSS (Sassy CSS) is a CSS preprocessor — a scripting language that extends plain CSS with programming-like features, and then compiles down into ordinary CSS that the browser can understand. It keeps the familiar CSS syntax but adds capabilities that CSS alone does not have.

How SCSS differs from plain CSS, and why Ascend uses it:

- **Variables:** SCSS lets you define reusable values (colours, spacing, font sizes) once and reference them everywhere, whereas plain CSS custom properties

are far more limited in scope and syntax.

- **Nesting:** Selectors can be nested inside one another to mirror the structure of the component itself, instead of writing long, repetitive selector chains as plain CSS requires.
- **Mixins and functions:** Reusable blocks of styling logic (for example, a mixin for a card shadow or a flex layout) can be written once and included wherever needed.
- **Partials and imports:** Styles can be split into small, focused files (one per component) and imported together, keeping the styling for a large project organised instead of one giant stylesheet.
- **Control directives:** Loops like `@for` and conditionals are available directly in the stylesheet — this is exactly how the staggered list and avatar animations in Ascend are generated, using an `@for` loop to assign an increasing animation-delay to each child element.

Because every component in Ascend has its own visual identity but still needs to share a consistent theme, SCSS's variables, nesting, and mixins make it far easier to keep styling modular, consistent, and maintainable than plain CSS would allow at this scale.

2.6.3 MUI (Material UI) Components

MUI is a popular React component library that implements Google's Material Design system as ready-made, production-quality components. Rather than building every button, dialog, or data table from scratch, the Ascend project imports these components directly from MUI and customises them to match its own theme.

I spent time going through MUI's official documentation to understand what it

offers. The documentation is organised around a component API reference for every component (listing all available props, default values, and usage examples), a separate theming section that explains how to override MUI's default colours, typography, and spacing to match a custom brand, and a "system" section covering the `sx` prop and the `styled()` utility, which let you apply custom CSS-in-JS styling directly onto any MUI component without writing a separate stylesheet. It also documents a large icon library and templates for common patterns like dashboards and forms.

In the Ascend project specifically, MUI is used for:

- Buttons, form inputs, and dropdowns used across the various pages.
- Dialog and Modal components for confirmation pop-ups and detail views.
- Snackbar components for toast-style notifications after an action succeeds or fails.
- AppBar, Card, and Chip components used to build the dashboard's layout and status indicators.

2.7 Practical Implementation: Intro Animation for the Ascend Project

Building on everything learned above, I implemented an introductory animation for the Ascend dashboard using React (.tsx), SCSS keyframe animations, and MUI components for pop-ups and notifications. The goal was to make the interface feel smoother and more polished on first load, rather than having every element appear instantly and abruptly. Below are four unique animation effects I implemented, along with exactly where each one was added.

- **1. Blur Reveal — applied to the entire dashboard container on initial page load.** The whole dashboard fades in from a blurred, slightly-lower position to a sharp, fully-visible state (opacity 0 → 1, filter: blur(8px) → blur(0), combined with a small upward translateY). This gives the page a premium "coming into focus" feel the moment a user opens the app, instead of the dashboard just popping into view.

- **2. Float + Fade — applied to the KPI summary cards** (Current Version, Next Version, Open Issues, and Sprint Progress) at the top of the dashboard. Each card animates upward slightly while fading in (`translateY(12px) → 0`, `opacity 0 → 1`), so the key metrics feel like they are settling gently into place rather than appearing all at once.

- **3. Sweep Reveal — applied to the charts and graphs section.** Using a clip-path animation, each chart draws itself from left to right instead of appearing instantly, which makes the data feel like it is actively being plotted for the user rather than just dumped on screen. This was the effect I liked best for the dashboard's analytics section.

- **4. Draw Line — applied to the project timeline / roadmap widget.** The connecting line between timeline milestones animates its width from 0% to 100%, so the roadmap visually "draws itself" left to right, reinforcing the sense of progress being mapped out rather than the whole timeline appearing statically. \

In addition to these SCSS-driven entrance animations, MUI's Dialog and Snackbar components were used to show pop-up confirmations and toast notifications during key actions on the dashboard, keeping the interaction feedback consistent with the same Material Design language used throughout the rest of the application. Together, the SCSS animations and MUI components made the dashboard's first impression noticeably smoother and more professional, while still keeping the codebase organised the same way I had learned from studying the rest of the Ascend project.

3. Vordenk — Digital Fleet Fingerprint

3.1 Introduction to Vordenk:

Vordenk is an independent project conceptualized and developed to address a specific, recurring gap in vehicle fault diagnosis: when a Diagnostic Trouble Code (DTC) fires on a vehicle, the maintenance team today sees only a snapshot at the exact moment of failure — a single reading of RPM, temperature, load, and speed. Two vehicles can throw the identical fault code yet because only the moment-of-fault snapshot is available, both are diagnosed and repaired the same way. This often leads to incorrect fixes and repeat failures, since the actual story behind the fault .

Vordenk builds a "fault fingerprint" for each DTC — how RPM, temperature, load, and speed behaved in the 10–30 minutes before the fault, drawn from every recorded instance across the fleet, not just one. Instead of seeing only "RPM = 2840 when P0301 fired," a mechanic sees the pattern behind it, giving diagnosis real context — much like how Vordenk, the "Fleet Fingerprint," treats each fault as unique rather than interchangeable.

The transition from observing ASCEND to building Vordenk was deliberate, not a separate exercise.

ASCEND's intro animation module gave hands-on experience with **React.js** and **Material UI (MUI)** .

This wasn't theoretical — it came from contributing directly to a live codebase.

It built real understanding of component organization, UI state management, and how a polished frontend comes together in practice.

3.2 Fundamentals of Diagnostic Trouble Codes (DTCs):

A Diagnostic Trouble Code is a standardized signal stored by a vehicle's Electronic Control Unit (ECU) whenever onboard sensors detect a reading outside normal operating limits. Rather than surfacing only a generic warning like the Check Engine Light, a DTC identifies the specific system and component responsible – engine, transmission, braking, emissions, or electrical – allowing a technician to move directly to the likely fault area instead of inspecting the vehicle blind. Codes are read through an OBD-II scanner or an OEM diagnostic tool, and in commercial/heavy-duty fleets, through the J1939 protocol used alongside OBD-II

Diagnostic Trouble Codes are grouped into categories based on the vehicle system where the fault occurs.

Powertrain Codes: Codes starting with P relate to engine, transmission, and emission control systems. Mechanics encounter these codes most often because they directly affect vehicle performance and fuel efficiency.

Body Codes: Codes beginning with B refer to components inside the vehicle body. Airbags, seatbelts, climate control systems, and lighting modules commonly fall under this category.

Chassis Codes: Codes marked with C indicate issues within chassis-related systems. Steering, suspension, braking systems, and traction control components typically generate these codes.

Network Codes: Codes starting with U point to communication problems between electronic control modules. Faults in the vehicle's internal network can prevent modules from sharing critical data properly.

3.2.1 How Is a DTC Code Structured?

Each Diagnostic Trouble Code (DTC) follows a standardized five-character format

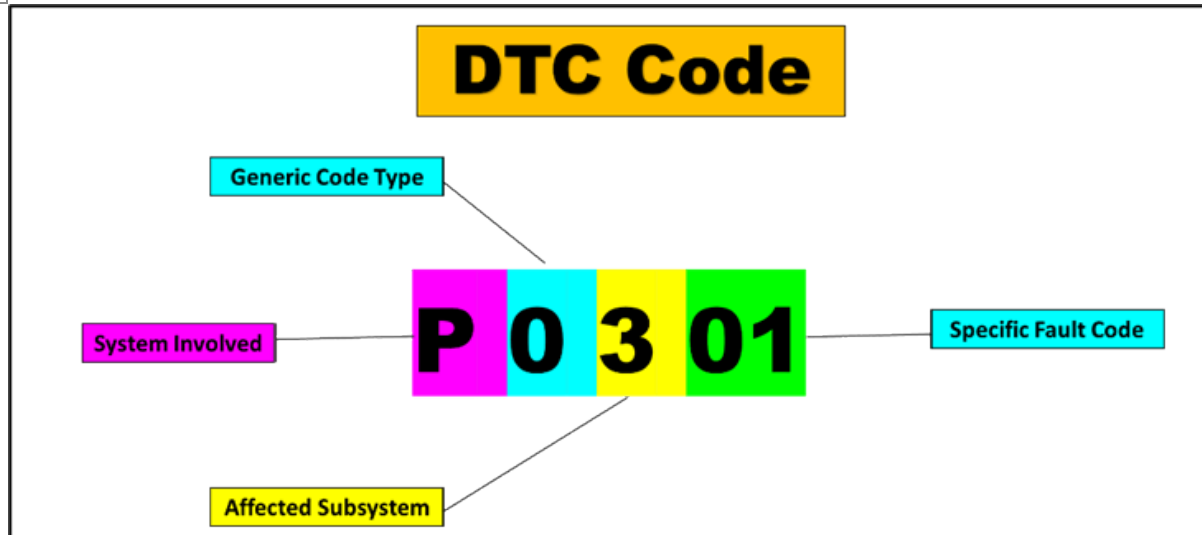
designed to identify the affected vehicle system and the specific malfunction detected by the onboard diagnostic (OBD) system. This standardized structure allows technicians and vehicle owners to quickly interpret fault codes, streamline the diagnostic process, and accurately identify issues requiring inspection or repair.

System Letter — The first character identifies the vehicle system: P for Powertrain, B for Body, C for Chassis, and U for network communication.

Code Type — The second digit shows whether the code is generic or manufacturer-specific: 0 for standardized OBD-II codes, 1 for manufacturer-defined codes.

Subsystem Area — The third digit narrows the issue to a specific subsystem, such as fuel delivery, ignition, emissions, or transmission, helping technicians focus on the right area.

Fault Identifier — The fourth and fifth digits describe the exact malfunction, pointing to the specific component, circuit, or condition responsible.



3.2.2 How to Read Diagnostic Trouble Codes?

Reading a Diagnostic Trouble Code involves breaking down the five-character code to understand which system is affected and what type of fault has been detected. Technicians interpret each character sequentially to narrow the issue to a specific vehicle subsystem or component.

1. Identify System

Start with the first letter, which indicates the affected system — **P** for powertrain, **B** for body, **C** for chassis, and **U** for communication network issues.

2. Check Code Type

The second digit shows whether the code is standardized or manufacturer-specific — **0** for generic OBD-II codes, **1** for manufacturer-defined codes.

3. Locate Subsystem

The third character narrows down the subsystem involved. In powertrain codes, for instance, this may point to fuel systems, ignition components, emission controls, or transmission functions.

4. Interpret Fault

The final two digits describe the exact fault detected by the control module. Technicians cross-reference these numbers with diagnostic databases or scan tool descriptions to identify the specific component or condition responsible.

5. Review Example

A code like **P0301** indicates a powertrain fault involving a cylinder-one misfire. Breaking down each character this way helps technicians quickly identify the affected system and begin targeted inspection.

Each character represents a different diagnostic layer:

Position	Meaning	Details / Example
1st Character	System Involved	P = Powertrain (engine, transmission) B = Body (airbags, AC, seats) C = Chassis (ABS, steering, suspension) U = Network/Communication (CAN, LIN issues)
2nd Character	Generic Code Type	0 = Generic (SAE/OBD-II standard) 1 = Manufacturer-specific 2/3 = Reserved for future use
3rd Character	Affected Subsystem	Identifies which part is affected: Example: For powertrain, 1 = Fuel & Air Metering, 3 = Ignition system
4th & 5th Characters	Specific Fault Code	Specific issue/failure detected. Example: P0301 → Cylinder 1 misfire detected P0420 → Catalyst efficiency below threshold

Example — P0301 indicates a powertrain fault related to a cylinder misfire, showing how the structure lets technicians quickly locate the issue and begin targeted inspection

3.2.3 What to Look for in a Vehicle Diagnostic Tool

Choosing the right diagnostic tool helps retrieve and interpret Diagnostic Trouble Codes accurately during vehicle maintenance and troubleshooting.

Vehicle Compatibility — The tool should support OBD-II protocols and work across multiple vehicle brands and models, ensuring it can read both generic and manufacturer-specific DTC codes.

Real-Time Data — Advanced scanners provide live sensor data from the vehicle's control modules, helping technicians diagnose intermittent or performance-related issues.

Code Database — A strong built-in code library translates DTCs into readable fault descriptions, letting technicians understand each code without searching external references.

User Interface — Clear displays and intuitive menus improve usability, allowing technicians to quickly navigate code lists, sensor data, and

troubleshooting information.

3.2.4 Why Fault Code Analysis Matters for Fleet Management

For a commercial fleet operator, DTCs are not just a repair-shop tool — they're a cost and uptime lever:

Faster diagnosis: technicians move straight to the affected system instead of trial-and-error inspection.

Lower maintenance cost: correctly identifying the root cause the first time avoids repeat repairs and repeat vehicle downtime.

Regulatory compliance: emission-related P-codes are mandatory under OBD-II, and in India, tied directly to BS-VI norms.

Fleet-wide visibility: a single vehicle's fault history is useful, but a fleet manager needs patterns across vehicles — which is exactly the gap Vordenk is built to close, since a snapshot-only fault code (as read by a standard scanner) still doesn't show why a fault happened, only that it happened.

3.2.5 Common Examples of Diagnostic Trouble Code

While DTCs can point to thousands of possible issues across a vehicle's systems, a handful of codes show up far more often than the rest — largely because they relate to common engine, emissions, and sensor faults that develop naturally over a vehicle's lifetime. Understanding these frequent codes gives technicians a practical starting point, since recognizing them on sight speeds up diagnosis before deeper investigation is needed. The examples below illustrate some of the most commonly encountered DTCs and what each one typically signals.

DTC Code	System Area	Meaning
P0300	Powertrain	Random or multiple cylinder misfire detected.
P0301	Powertrain	Misfire detected in cylinder 1.
P0171	Fuel System	System running too lean due to incorrect fuel-air mixture.
P0420	Emissions	Catalyst system efficiency below threshold.
P0442	EVAP System	Small leak detected in the evaporative emission system.
P0128	Cooling System	Engine coolant temperature below expected operating range.
P0500	Speed System	Vehicle speed sensor malfunction detected.

3.3 Problem Statement :

When a **Diagnostic Trouble Code (DTC)** is triggered in a commercial vehicle, the diagnostic system records only a **freeze-frame snapshot** of parameters such as RPM, engine temperature, vehicle speed, and engine load at the exact moment the fault occurs. While this identifies **what** fault was detected, it provides no information about **how the vehicle was behaving before the fault**, leaving technicians without the operational context needed for accurate diagnosis.

The same DTC can result from different underlying conditions, such as a genuine mechanical failure or temporary factors like overheating, excessive load, or prolonged operational stress. Vordenk bridges this gap by identifying the root cause of faults, enabling more accurate diagnostics, reducing unnecessary maintenance, and improving fleet reliability.

Consider a truck reporting **P0301 (Cylinder Misfire)**. The technician replaces the spark plug based on the fault snapshot, but the issue returns because the actual cause was overheating under sustained engine load. Conventional diagnostics capture only the fault moment, while Vordenk analyzes the events leading up to

it to identify the true root cause.

3.4 Impact of this gap on the vehicle and the fleet:

Misdiagnosis at the point of repair — A technician sees "P0301, cylinder misfire" and defaults to the most common cause (e.g., spark plug), without knowing that engine load had been climbing for 18 minutes and coolant crossed a critical threshold beforehand. The actual cause — a cooling system issue causing misfire under load — goes untreated.

Repeat failures and repeat downtime — Because the wrong component gets replaced, the vehicle returns to the workshop for the same fault within weeks, doubling repair time and taking the vehicle off the road twice for one underlying issue.

No visibility across the fleet — A single mechanic in front of one truck has seen that fault maybe 2–3 times in their career. The fleet as a whole may have thrown that same code 40+ times across different vehicles — but that collective pattern is never captured or shared, so every technician starts from zero each time.

Rising maintenance cost per vehicle — Wrong-part replacement, repeat labour, and repeat vehicle recovery/towing all stack up as avoidable cost, purely because the diagnosis was based on a snapshot instead of a trend.

Reduced fleet uptime — For a commercial fleet running on tight delivery/logistics schedules, every unplanned or repeat breakdown directly affects route reliability and operational cost — not just the cost of the part.

Safety risk in critical systems — For chassis- or brake-related fault codes (C-codes) in particular, a misdiagnosed root cause means the actual safety-critical issue may remain unresolved even after a "repair" is logged as complete.

No institutional memory of past faults — Once a case is closed, the

reasoning behind it is rarely recorded in a form that helps the next technician who sees the same code on a different vehicle — so lessons are not retained at the fleet level.

Even a conservative 15–20% reduction in repeat-repair incidents across a fleet of this scale translates directly into measurable savings in labour hours, part costs, and vehicle downtime making this a cost-of-operations problem, not merely a technical inconvenience.

3.4.1 Proposed Solution:

The same P0301 case from Section 3.3 illustrates exactly what changes once Vordenk is introduced into the diagnostic process.

Revisiting the P0301 case (Section 3.3):

Under the current system, the technician saw a single frozen reading — RPM 2840, temperature 97°C, load 84%, speed 62 km/h — with no way of knowing what led up to it. The spark plug was replaced. The fault returned two weeks later.

Now, place that same case inside Vordenk. The moment P0301 fires, Vordenk doesn't just log the snapshot — it pulls engine data from the 10–30 minutes before the fault and presents it as a clear trend:

"Engine load climbed steadily for 18 minutes before the fault. Coolant crossed 94°C at the 12-minute mark. RPM fluctuated unusually in the final 5 minutes. Speed stayed normal throughout."

This single insight changes the diagnosis entirely:

Diagnosis

Before	
Vordenk	"Cylinder misfire →check spark plug."

With "Load and heat were the real stressors, not ignition — this is a
Vordenk cooling system issue manifesting as a misfire under load."

The result: the right fix, applied the first time — because the technician finally sees the build-up, not just the breaking point.

In short: every DTC gets a **fault fingerprint** — the pre-fault trajectory of RPM, temperature, load, and speed, built from every recorded instance of that fault across the fleet. A raw scanner gives a technician a moment. Vordenk gives them a pattern.

3.4.2 How Vordenk Solves the Problem:

Problem	How Vordenk Resolves It
Misdiagnosis at the point of repair	Surfaces the pre-fault trend alongside the fault code, so diagnosis is based on actual behaviour — not a guess at the most common cause.
No visibility across the fleet	Every fingerprint is built from all vehicles that threw that code, not just the one in the workshop — the technician inherits the fleet's collective experience.
No institutional memory	Every case — recording, analysis, report, and outcome — is logged against the vehicle's ID as searchable history, so the next technician starts with context, not from zero.
Repeat failures, rising cost, reduced uptime	All three are downstream effects of one root cause — wrong diagnosis. Fixing the diagnosis at the source resolves all three together.

Vordenk does not remove human judgment from the process at any stage — data is still recorded, findings are still reviewed with the ability to override the AI's suggestion, and fixes are still physically confirmed before a case is closed. What changes is not who makes the decision, but what information is available to them when they make it.

3.5 Central Work Idea

From the diagnostic insights of Vordenk, this section explains the machinery behind it — how a single fault case actually moves through the system, from the moment a vehicle is connected for recording to the moment a case is closed. Vordenk is built around three distinct roles working in a fixed sequence, with every case threaded together by one constant: the vehicle ID.

3.5.1 System Workflow

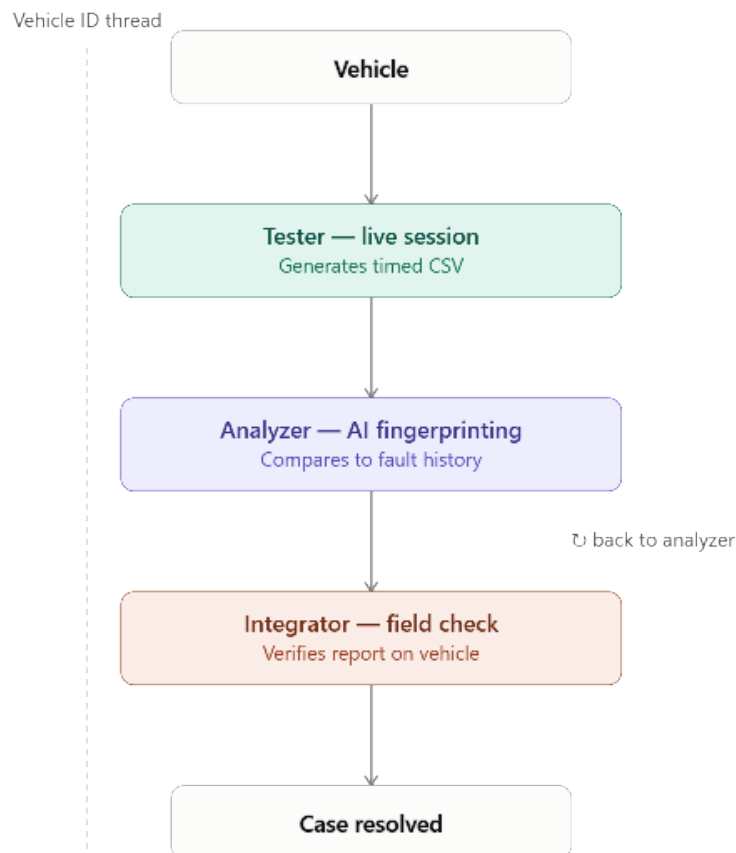
At its core, Vordenk treats fault diagnosis as a **relay, not a single task** — no one role is expected to record, analyse, and verify a fault alone. Instead, each role owns one stage of the process and hands off a structured output to the next, so that by the time a case reaches its final decision, it has passed through three independent checks rather than one person's judgment.

Tester — physically connects to the vehicle and records live engine parameters (RPM, temperature, load, speed) over the pre-fault window, then sends the completed recording forward.

Analyzer — takes that recorded data, identifies what deviated before the fault fired, and drafts a structured report with a likely cause and fix.

Integrator — receives the report, physically verifies the diagnosis on the vehicle itself, and either confirms and closes the case, or sends it back with field notes if the diagnosis doesn't hold up.

Every one of these handoffs is tracked under a single vehicle ID, so the recording, the analysis, the report, the messages exchanged between roles, and the final outcome all sit on one continuous timeline — nothing gets lost between stages, and any role can trace a case back to its origin at any point.



3.5.2 Role-Based Workflow:

Vordenk follows a three-role workflow to ensure that vehicle diagnostic data is collected, analyzed, and verified in a structured manner. Each role receives specific inputs, performs designated tasks, and produces outputs that are passed to the next stage.

Tester

Input: A vehicle ID (entered or selected) and the live engine parameter stream from the vehicle — RPM, temperature, load, and speed, read continuously as the vehicle operates.

Process:

- o Starts recording; parameters stream in and are plotted live as the session runs.

- o The system continuously watches for a DTC event; once a fault fires, the pre-fault window (10–30 minutes prior) is what matters most for diagnosis.
- o On Stop, the recorded session is automatically compiled into a CSV file.
- o A rule-based completeness check runs on that CSV — confirming how much of the pre-fault window was actually captured, and flagging any gaps.

Output: A completed session summary (duration, whether a fault fired, completeness score) is shown to the Tester for confirmation, then the CSV, DTC code, and vehicle ID are sent to a specific Analyzer chosen from the team roster — opening a shared message thread for that case.

Analyzer

Input: The Tester's compiled CSV, the DTC code, and the vehicle's prior history, received into an incoming queue sorted by time.

Process:

- o Opens the CSV data as a trajectory chart — RPM, temperature, load, and speed plotted across the pre-fault window.
- o The AI Insight Panel reads the trend data, the DTC code, and past vehicle history, and returns a likely root cause and recommended fix with a confidence score.
- o The Analyzer accepts, edits, or overrides this suggestion — the AI supports the diagnosis, it does not finalize it.
- o Findings are compiled into a structured report: issue, evidence, recommended fix, and relevant history.

Output: A reviewed, edited report is sent to a specific Integrator chosen from the roster, continuing the same vehicle-ID thread.

Integrator

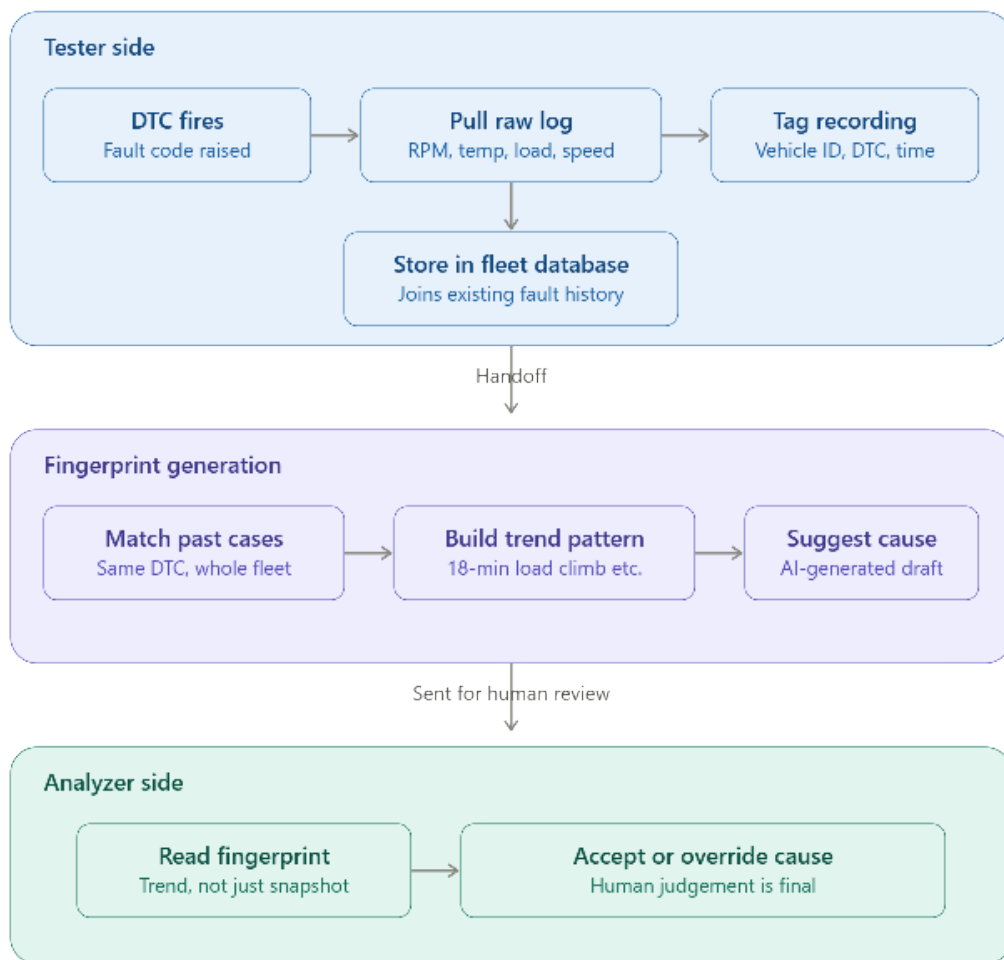
Input: The Analyzer's structured report, displayed exactly as written — read-only, with no reinterpretation at this stage.

Process:

- o A field verification checklist is auto-generated from the specific claims made in the report.
- o The Integrator physically inspects the vehicle against that checklist and records the actual findings.

Output: One of two outcomes:

- o **Confirmed:** the case is closed, and the outcome is logged permanently to the vehicle's history.
- o **Disputed:** the case is sent back to a specific Analyzer with field notes attached, and the thread stays open until re-resolved.



3.5.3 Feature Mapping per Role

The three roles don't operate as isolated modules — each one's output is structured specifically to be the next role's input, so the pipeline stays coherent end to end:

Stage	Tester	Analyzer	Integrator
Primary action	Records live engine data	Diagnoses the fault	Physically verifies the fix
Input received	Vehicle ID / physical connection	Tester's recording + completeness score	Analyzer's structured report
Tool	Live recording	Trajectory chart, AI Insight	Pre-populated

support	chart, completeness check	Panel	field checklist
Handoff	Sends to a chosen Analyzer	Sends to a chosen Integrator	Closes case, or sends back to a chosen Analyzer
Shared thread	Message thread opens with Analyzer	Same thread continues with Integrator	Thread stays open if case is disputed

This mapping is what keeps the system accountable end to end — every stage produces a defined artifact (a recording, a report, a checklist outcome) rather than a verbal handoff, and every artifact is tied to the same vehicle ID, so the complete diagnostic story for any vehicle can be reconstructed at any time, by any role.

3.6 Technology Stack

Vordenk's stack was chosen to carry forward the frontend foundation built during the ASCEND observation phase, while adding a backend and AI layer suited to an independent build without a dedicated backend team. The diagram above shows how the three layers communicate — the frontend handles all requests and data entry, Supabase manages authentication, storage, and persistence, and the Lovable AI gateway sits behind Supabase to generate the fault analysis itself.

3.6.1 Frontend — React, Vite, SCSS

React.js was used as the core UI framework, initiated the same way it was first learned during ASCEND — structuring the interface as independent, reusable components (dashboard cards, recording views, report panels) rather than one monolithic page.

Vite was used as the build tool and development server, chosen over a traditional bundler for its fast hot-module-reload during development, which sped up the iteration cycle while building and testing UI components.

JavaScript (ES6+) was the primary implementation language — used for component logic, state handling, and API calls to Supabase.

SCSS was used for styling, initiated as an extension of plain CSS to allow variables, nesting, and reusable style partials across components — keeping the visual language (colors, spacing, typography) consistent across every page instead of repeating styles per component.

Connectivity: Every component built in React calls out to Supabase directly through its client library for data operations (fetching sessions, sending recordings, retrieving reports), so the frontend never talks to the AI gateway directly — all AI-related requests are routed through Supabase first.

3.6.2 Backend — Supabase

Supabase was chosen as the backend layer to avoid building and maintaining a custom server from scratch, since it provides a managed Postgres database, authentication, and file storage in one integrated platform.

Database stores all structured records — vehicle IDs, sessions, DTC codes, analysis results, and report data — in Postgres tables.

Authentication handles role-based login (Tester, Analyzer, Integrator), so each user only sees the pages and data relevant to their role.

Storage holds uploaded recording data (CSV files) generated during the Tester's recording sessions.

Connectivity: Supabase acts as the single point of coordination between the frontend and the AI layer — it receives requests from React, stores or

retrieves the relevant data, and forwards fault-analysis requests to the Lovable AI gateway, then stores the returned result back in the database for the frontend to display.

3.6.3 AI Layer — Lovable AI Gateway

The Lovable AI gateway was chosen as the AI processing layer to generate the root-cause suggestions and report narratives described in Analyzer role, without needing to host or manage a custom model backend.

It receives the structured pre-fault trend data, the DTC code, and relevant vehicle history (passed through Supabase), and returns a likely root cause, a suggested fix, and a confidence score.

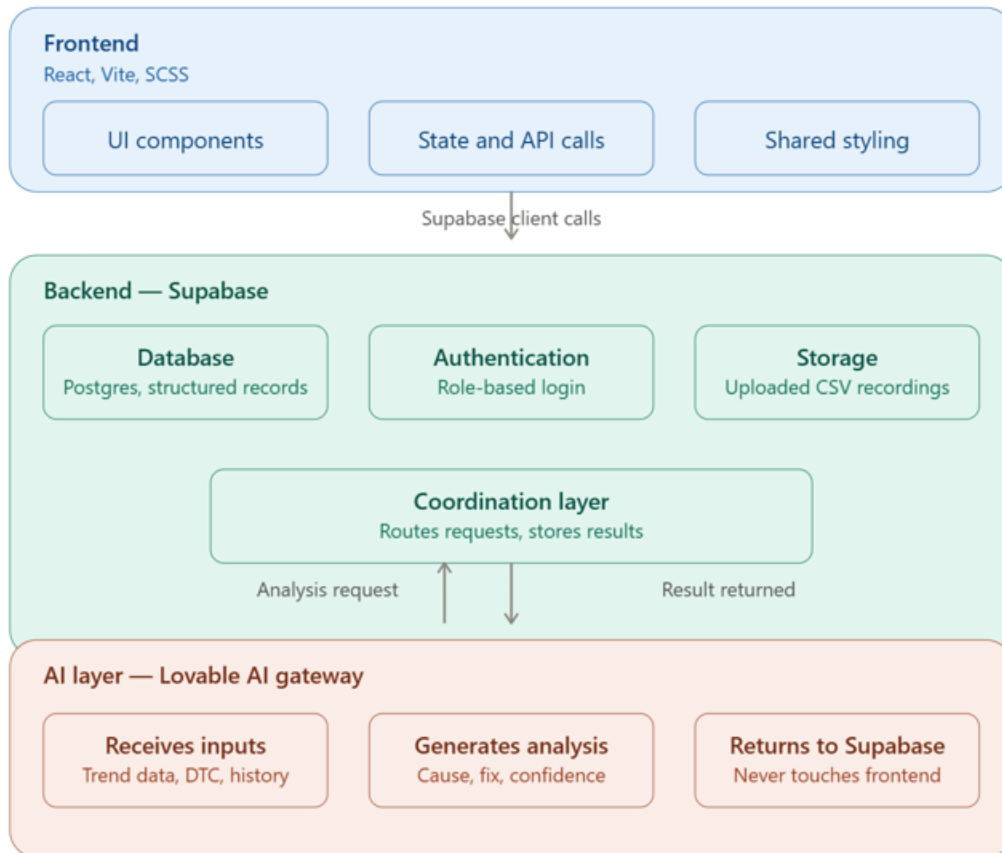
Connectivity: The gateway never communicates with the frontend directly — every request to it originates from Supabase, and every result it returns is written back into Supabase before the frontend ever displays it. This keeps the AI layer decoupled from the UI and centralizes all data flow through one backend.

3.6.4 Why This Stack Was Chosen

React + Vite + SCSS was a direct continuation of the frontend skills built during ASCEND — using a familiar, proven approach meant more time could go into Vordenk's actual features rather than relearning a framework.

Supabase removed the need to build and secure a custom backend independently, letting the project focus on the diagnostic workflow itself rather than infrastructure.

Lovable AI gateway provided AI capability without requiring a separately hosted model or API key management from scratch, keeping the build achievable within the scope of an independent project.



3.7 Implementation :

React Implementation — Component Structure & Page Control

The application is structured around a **role-based page hierarchy** — each of the three roles (Tester, Analyzer, Integrator) has its own set of pages, with a shared Dashboard, Vehicle History, and Messaging component reused across all three.

Routing controls which pages a user can access based on their logged-in role — a Tester is routed only to Record Vehicle, My Recordings, and Team pages, while an Analyzer or Integrator sees their own equivalent set. This prevents a user from manually navigating into a page meant for a different role.

Component-per-page organization was followed — each page (e.g., Record Vehicle, Incoming Queue, Report Builder) is built as its own component file, with shared UI elements (Sidebar, Top Bar, Notification Bell, Message Thread) extracted into reusable components rather than repeated per page.

State management is handled at the component level using React's built-in state (useState), with data fetched from Supabase on page load and re-fetched or updated after key actions (e.g., after a recording is sent, the My Recordings list refreshes automatically).

Conditional rendering drives most of the interface — for example, the Record Vehicle page shows a "Start Recording" button by default, then switches to a live status view (Connected / Recording / Fault Detected) once recording begins, and finally a review summary once recording stops — all within the same page component, controlled by state rather than separate routes.

Navigation flow between pages follows the actual case lifecycle — sending a recording from the Tester's Team page, for instance, redirects into the shared Vehicle History view so the sender can immediately see the case now logged against that vehicle ID.

3.7.1 SCSS Implementation

SCSS was structured using **partials** — separate .scss files for variables (colors, spacing, typography), shared mixins, and page-specific styles — imported into a single main stylesheet rather than writing one large CSS file.

Variables were used for the core design tokens (primary colors, font sizes, spacing units), so a single change (e.g., adjusting the primary accent color) updates every component using that variable, instead of manually editing multiple files.

Nesting was used to scope styles to their parent component structure — for example, styles for a dashboard card's title, subtitle, and icon are nested inside the card's own selector block, keeping related styles grouped and readable.

Reusable mixins were written for repeated patterns — such as button styles, card shadows, and responsive breakpoints — so the same visual treatment could be applied across the Tester, Analyzer, and Integrator pages without duplicating code.

This structure directly extended the SCSS practices first studied during the ASCEND observation phase, applied here across a larger, multi-role interface instead of a single component.

3.7.2 Supabase Implementation

Database tables were created for each core entity — users, vehicles, sessions, analyses, reports, and messages — mirroring the roles and case lifecycle described in Section 3.5, with each table linked by foreign keys (e.g., every session tied to a vehicle ID and a Tester ID).

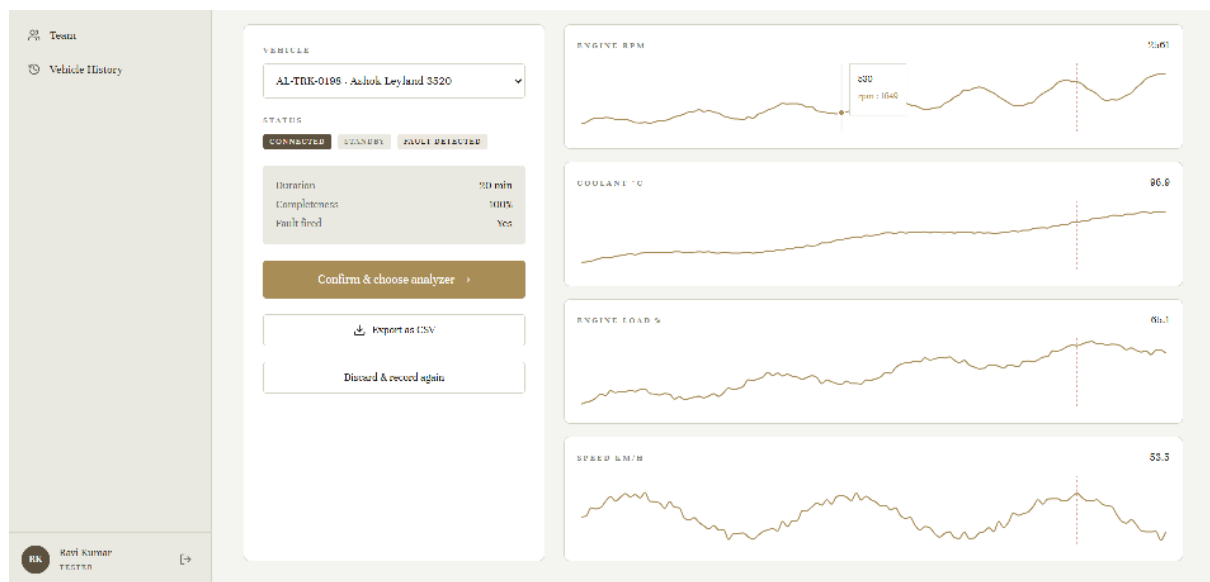
Real-time data fetching — pages like Incoming Queue and Vehicle History pull directly from Supabase tables on load, so a newly sent recording or report appears for the receiving role without needing a page refresh cycle beyond the standard fetch.

File storage — recorded CSV files generated from the Tester's recording sessions are uploaded to Supabase Storage, with the resulting file reference stored in the corresponding session record so it can be retrieved later by the Analyzer.

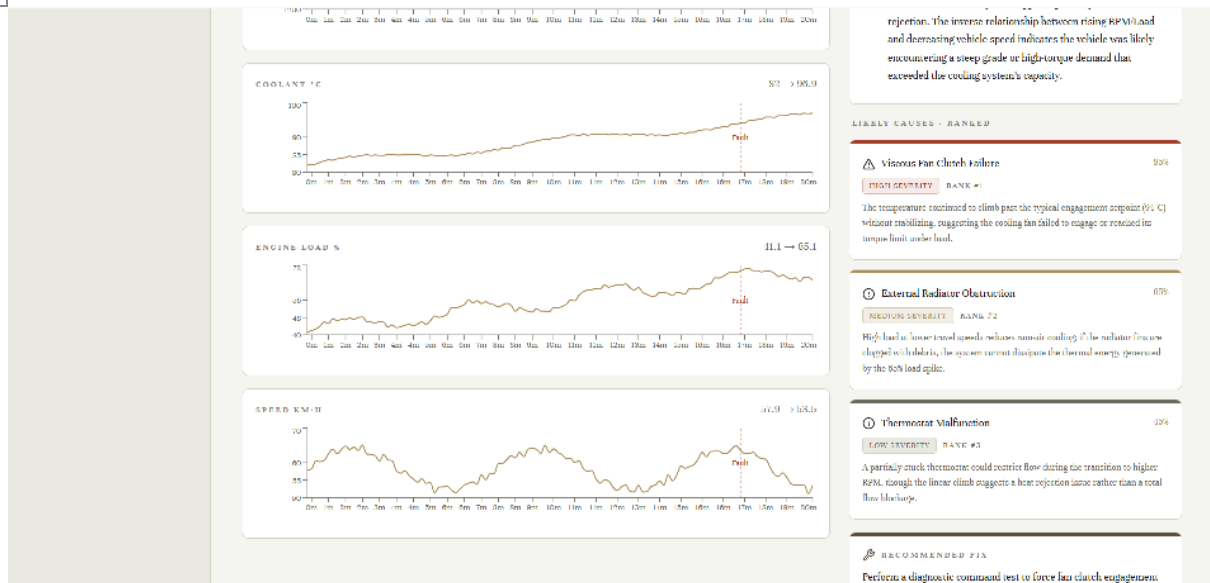
Row-level access control — queries are scoped so a Tester only retrieves their own sessions, an Analyzer only retrieves cases sent to them, and so on, enforcing the same role boundaries at the data layer that are already

enforced in the UI.

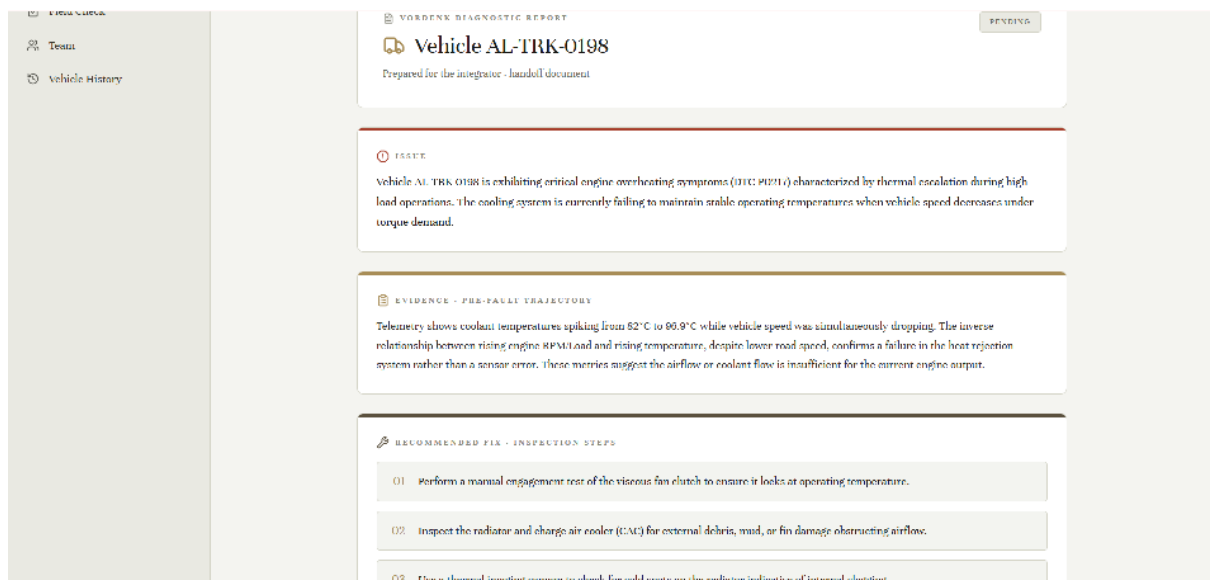
Connectivity to the AI layer — when an Analyzer requests an AI insight, the frontend calls Supabase, which forwards the relevant session data (trend summary, DTC code, vehicle history) to the Lovable AI gateway, receives the response, and writes it back into the analyses table for the frontend to display.



DTC fired at the Tester level using the datasets



With this raised DTC Analyzer creates a report using AI



The Report from the Analyzer to the Integrator

Incoming Reports

Report Review

Field Check >

Team

Vehicle History

CHECKLIST

☐ Confirmed reported issue on vehicle: Vehicle AL-TM0-0198 is exhibiting critical engine overheating symptoms (DTC P0217) characterized by thermal escalation during high-load operations. The cooling system is currently failing to maintain stable operating temperatures when vehicle speed decreases under torque demand.
 ☐ Performed / verified — Perform a manual engagement test of the viscous fan clutch to ensure it locks at operating temperature.
 ☐ Performed / verified — Inspect the radiator and charge air cooler (CAC) for external debris, mud, or fin damage obstructing airflow.
 ☐ Performed / verified — Use a thermal imaging camera to check for cold spots on the radiator indicative of internal clogging.
 ☐ Performed / verified — Test the thermostat opening temperature and verify coolant level and concentration.
 ☐ Performed / verified — Pressure test the cooling system for leaks or a faulty radiator cap that may be lowering the boiling point.
 ☐ No visible fluid leaks observed under load.
 ☐ Fault reproduction attempted after inspection.

FIELD NOTES

REPORT SUMMARY

Issue: Vehicle AL-TM0-0198 is exhibiting critical engine overheating symptoms (DTC P0217) characterized by thermal escalation during high-load operations. The cooling system is currently failing to maintain stable operating temperatures when vehicle speed decreases under torque demand.

Fixes:

1. Perform a manual engagement test of the viscous fan clutch to ensure it locks at operating temperature.
2. Inspect the radiator and charge air cooler (CAC) for external debris, mud, or fin damage obstructing airflow.
3. Use a thermal imaging camera to check for cold spots on the radiator indicative of internal clogging.
4. Test the thermostat opening temperature and verify coolant level and concentration.
5. Pressure test the cooling system for leaks or a faulty radiator cap that may be lowering the boiling point.

Confirm & close case

Dispute & send back

From the Report Integrator starts the Fiel check

4. Challenges Faced

Building Vordenk independently meant navigating obstacles without the support structure of an ongoing team project — every architectural decision, integration issue, and design trade-off had to be worked through alone. The challenges below span technical, conceptual, and scope-related hurdles encountered across the build.

Data availability — Simulating realistic fleet-wide fault histories was difficult without access to real vehicle telemetry, requiring careful construction of representative sample data to test the fingerprint logic meaningfully.

Coordinating three layers independently — Connecting the frontend, Supabase, and the Lovable AI gateway without a dedicated backend team

meant debugging integration issues (auth flow, request routing, data shape mismatches) single-handedly.

Designing the fingerprint logic — Deciding what counts as a meaningful pre-fault pattern (which parameters, what time window, how much deviation matters) required iterating through multiple approaches before settling on a workable structure.

Role-based access control — Implementing distinct views and permissions for Tester, Analyzer, and Integrator roles within Supabase's authentication system required extra care to avoid data leakage between roles.

Balancing AI suggestion with human control — Ensuring the AI gateway's output supported the analyzer's decision-making rather than overriding it needed careful UI and workflow design, not just a technical integration.

Time and scope constraints — As an independent, single-person project, prioritizing which features to build first (core fingerprinting vs. polish, additional dashboards, etc.) meant constantly balancing ambition against what was achievable in the available time.

5. Future Scope

Vordenk in its current form proves the core concept — that pre-fault behaviour patterns can meaningfully improve diagnosis — but there is significant room to expand its capabilities, accuracy, and real-world readiness. The points below outline the direction the project could take beyond this initial build.

Real-world data integration — Connecting Vordenk to actual vehicle telemetry systems (OBD-II live streams) instead of simulated data, allowing the fingerprint logic to be validated and refined against genuine fleet behaviour.

Predictive alerts — Moving from reactive diagnosis (analyzing a fault after it fires) to predictive warnings, flagging vehicles whose current behaviour matches the early stages of a known fault fingerprint before the DTC actually triggers.

Cross-fleet and cross-manufacturer learning — Extending the fingerprint database beyond a single fleet to aggregate patterns across multiple fleets or vehicle manufacturers, improving fingerprint accuracy for rarer fault codes.

Mobile and offline support — Building a mobile-friendly or offline-capable version of the tester interface for use directly in the workshop or field, where connectivity may be limited.

Deeper AI explainability — Enhancing the Lovable AI gateway's output with more transparent reasoning (e.g. highlighting exactly which part of the trend influenced the suggested cause) to further build analyzer trust in the AI's suggestions.

Cloud-based Fleet Analytics Dashboard — A centralized cloud dashboard can be introduced to visualize fleet-wide performance, fault trends, and vehicle health scores. This would support fleet managers in making data-driven decisions and optimizing maintenance schedules across multiple vehicles.

Enhanced Cybersecurity and Data Protection Future versions should adopt stronger cybersecurity measures such as encryption and multi-factor authentication, along

6. Feedback

Structured Internship Orientation : Conducting a comprehensive orientation program at the beginning of the internship would help interns understand the

company's organizational structure, departmental functions, safety guidelines, and workflow, enabling them to adapt more quickly to the work environment.

Cross-Department Exposure : Providing interns with opportunities to visit and interact with multiple departments would enhance their understanding of the complete vehicle development process and improve their knowledge of interdisciplinary engineering practices.

Regular Technical Sessions : Organizing weekly or bi-weekly technical seminars and knowledge-sharing sessions by experienced engineers would expose interns to the latest automotive technologies, industry standards, and ongoing research activities.

Hands-on Project Involvement : Allowing interns to participate more actively in live projects and practical development activities would strengthen their technical skills, improve problem-solving abilities, and provide valuable real-world engineering experience.

Periodic Feedback and Review : Conducting regular review meetings between mentors and interns would help monitor progress, address technical challenges, provide constructive feedback, and ensure that learning objectives are effectively achieved.

7. References

The following resources were referred to during the course of this internship and project work:

1. **W3Schools** — Used as a reference for foundational web development concepts, including HTML, CSS, and JavaScript syntax. Available at: [https://](https://www.w3schools.com/)

2. **MUI (Material UI) Official Documentation** — Referred to for understanding the component API, theming, and styling system (the `sx` prop and `styled()` utility) used to implement Material UI components within the ASCEND project. Available at: <https://mui.com/>
3. Matrack Inc., "DTC Codes: Meaning, Types, and How to Read Diagnostic Trouble Codes" — Referred to for understanding the structure, classification, and interpretation of Diagnostic Trouble Codes, which formed the conceptual foundation for the Vordenk project. Available at: <https://matrackinc.com/dtc-codes/>
4. TuningBot, "DTC Codes Explained: OBD Fault Codes Guide" — Referred to for additional clarification on OBD-II fault code structure and common diagnostic trouble code examples. Available at: <https://tuningbot.com/dtc-codes-explained/>